

# Kasparro Shopper

A tool-calling shopping agent grounded in the Shopify Global Catalog, designed so that hallucinated products are structurally impossible rather than merely discouraged.

AUTHOR

Ridhi Kumari

SUBJECT

**Agentic AI**  
Autonomous tool-using systems

DOMAIN

**Agentic commerce**  
Conversational product discovery

## 01 Abstract

Kasparro Shopper turns a sentence such as *“a gaming laptop under \$1500 with good battery life”* into a defended purchase recommendation drawn from live, cross-merchant retail inventory. It is not a chatbot with a product database attached. It is an agent: it chooses which of eleven tools to invoke and in what order, observes each result, revises its own query when results come back wrong, and halts on an explicit stopping condition rather than when it runs out of things to say.

This report sets out the motivation for an agentic approach, the system architecture, the design of the agent's reasoning loop and tool interface, the mechanism that grounds its claims in real data, and an empirical evaluation of its behaviour. It closes with an honest account of where the system falls short.

|                                            |                                              |                                               |
|--------------------------------------------|----------------------------------------------|-----------------------------------------------|
| <div>11</div> <div>AGENT TOOLS</div>       | <div>10</div> <div>MAX REASONING STEPS</div> | <div>3</div> <div>COOPERATING PROCESSES</div> |
| <div>29</div> <div>SELECTABLE MODELS</div> | <div>8</div> <div>EVALUATION SCENARIOS</div> | <div>5 / 8</div> <div>FULLY CLEAN RUNS</div>  |

### The central idea

Most conversational shopping demonstrations allow the language model to write product recommendations as prose. The model may then invent a price, a specification, or an entire

product, and nothing in the system objects.

Kasparro Shopper inverts this. The interface renders product cards *only* from tool results. There is no path by which the model's own text becomes a product card. To show a laptop, the agent must successfully query the live catalogue and then present identifiers returned by that query. **Hallucinating a product is therefore not something the model is merely discouraged from doing; it is something the architecture does not permit.** Much of the rest of the design follows from holding that line.

---

## 02 Problem and motivation

Conventional retail search is keyword-shaped, whereas buyers are intent-shaped. A shopper who knows only that they want “something for editing 4K video on a budget” must translate that into keywords, then into filters, then compare a dozen tabs by hand. The translation is where they give up.

An agent is the appropriate architecture here because the task has the three properties that justify agency over a single model call:

---

|                     |                                                                                                                                                                                                                                                      |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unknown depth       | The number of steps required is not knowable in advance. A precise request may need one search; a vague one may need a clarifying question, a search, a refinement, and a comparison.                                                                |
| External truth      | The answer lives outside the model, in a catalogue that changes continuously. The model's own parametric knowledge is not merely insufficient here, it is actively harmful: a confidently recalled but outdated price is worse than no price at all. |
| Recoverable failure | Searches return empty or off-target results frequently. A system able to observe this and re-query outperforms one that simply reports failure to the user.                                                                                          |

---

Three concrete failure modes observed in existing tools shaped the design:

- **Stale recommendations in fast-moving categories.** A query for a gaming phone that surfaces a device several hardware generations old is a valid keyword match and a wrong real-world answer.
- **Comparison without a decision.** Specification tables that list every attribute and recommend nothing, leaving the buyer exactly where they started.
- **Unsupported factual claims.** Confident answers to compatibility questions with no evidence behind them.

## 03 Agentic design

“Agent” is an overloaded term, so this section assesses the system against the standard capability checklist for autonomous tool-using systems, and sets out how each capability is realised.

| CAPABILITY           | STATUS      | REALISATION                                                                                               |
|----------------------|-------------|-----------------------------------------------------------------------------------------------------------|
| Tool use             | IMPLEMENTED | Eleven tools with typed, validated inputs, bound to each conversation.                                    |
| Multi-step loop      | IMPLEMENTED | Up to ten reasoning steps per turn, each observing the previous tool result.                              |
| Stopping conditions  | IMPLEMENTED | A step budget, and a semantic stop when the agent asks the user a question.                               |
| Planning             | IMPLEMENTED | A nine-branch decision policy mapping intent shape to an opening action.                                  |
| Working memory       | IMPLEMENTED | Prior query, filters and results carried across turns, so follow-ups such as “cheaper” resolve correctly. |
| Grounding            | IMPLEMENTED | Interface elements render only from tool results. See §9.                                                 |
| Self-correction      | IMPLEMENTED | On empty or off-target results the agent re-queries with altered terms before reporting failure.          |
| Automated evaluation | IMPLEMENTED | A harness asserting on the agent's <i>action sequence</i> , not merely its prose. See §10.                |

### DESIGN POSITION

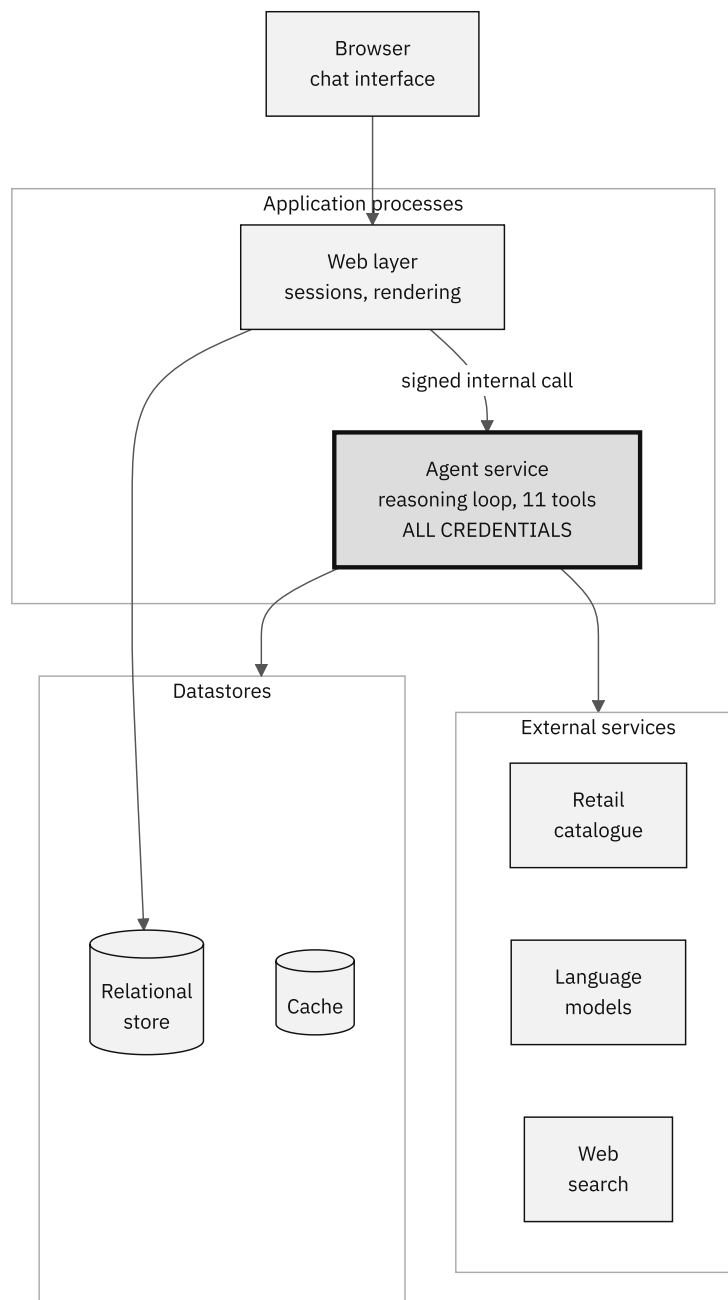
Planning is expressed as policy given to the model rather than as a separate planner component. This is a deliberate trade. A declarative policy is inspectable, adds no additional inference cost per turn, and can be evaluated directly by asserting on the resulting action sequence — which is precisely what the evaluation in §10 does. The cost is that the policy is only as reliable as the model's adherence to it, and the results show weaker models do drift.

## 04 System architecture

The system comprises three cooperating processes and two datastores. The separation is not decorative: the agent service is the *only* component holding credentials for the catalogue, the language models, or web search. The web layer never contacts those services directly, so the credential surface is confined to a single process.

FIGURE 1

Every credential-bearing call originates in the agent service; the browser and web layer hold none.



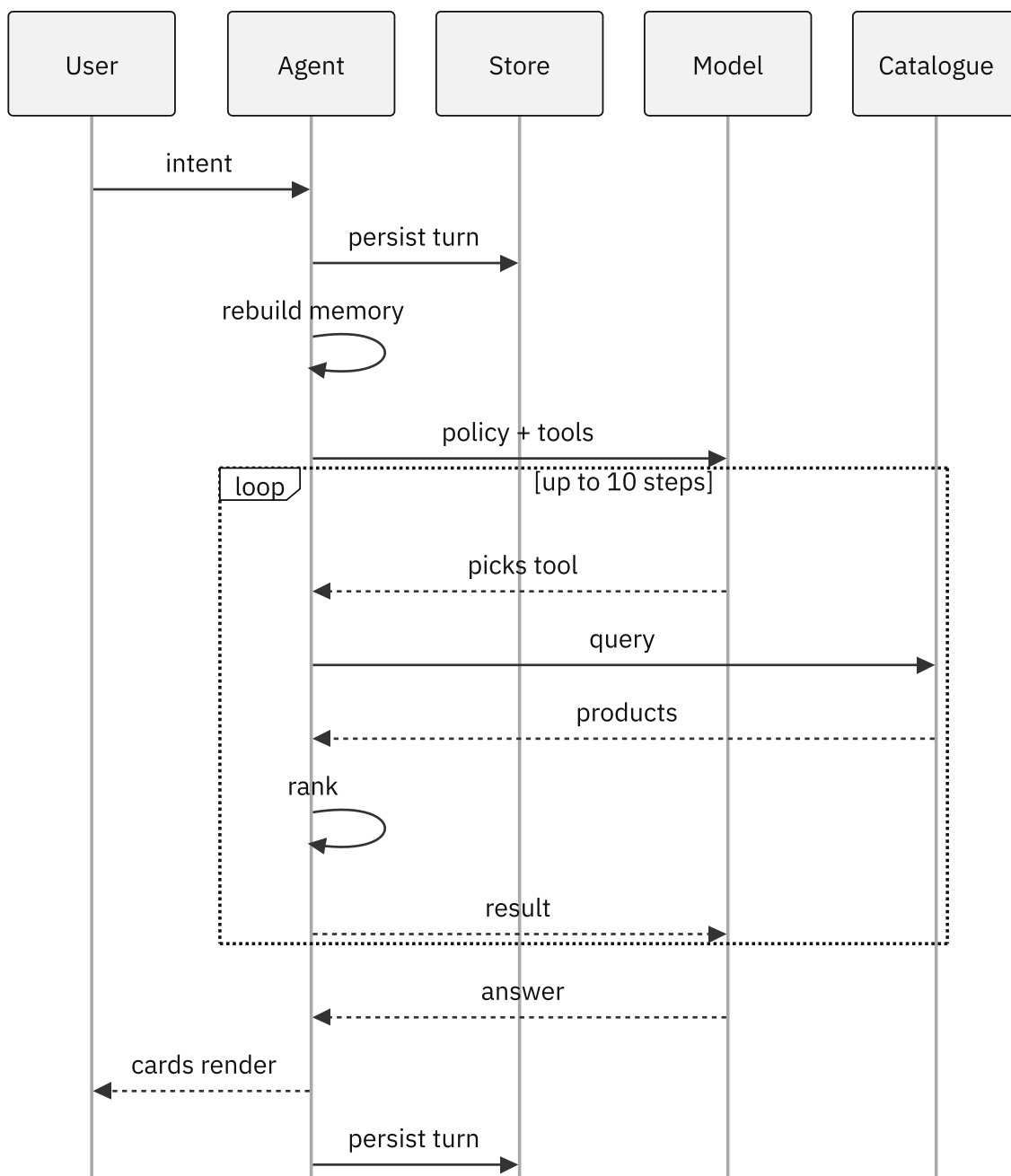
The agent service, drawn heavier, is the trust boundary of the system.

## 4.1 How one turn proceeds

The ordering of the two persistence operations is deliberate: the user's message is committed to storage *before* the response stream opens, so a dropped connection can never lose their input.

FIGURE 2

A single turn, from the user's message to a rendered product card, including the internal reasoning loop.



The boxed region is the agent loop. The model never sees raw network traffic; it sees typed results and decides the next action from them.

## 4.2 Trust boundary

Two independent checks protect the agent service. A shared secret authenticates the *calling process*, ensuring only the application's own web layer can drive the agent and consume inference budget. Separately, every request carries a user identity, and handlers verify that the requesting user owns the conversation before serving it. Passwords are stored as salted hashes, and guest sessions are real anonymous records so that guests and registered users travel identical code paths.

---

# 05 Agent behaviour

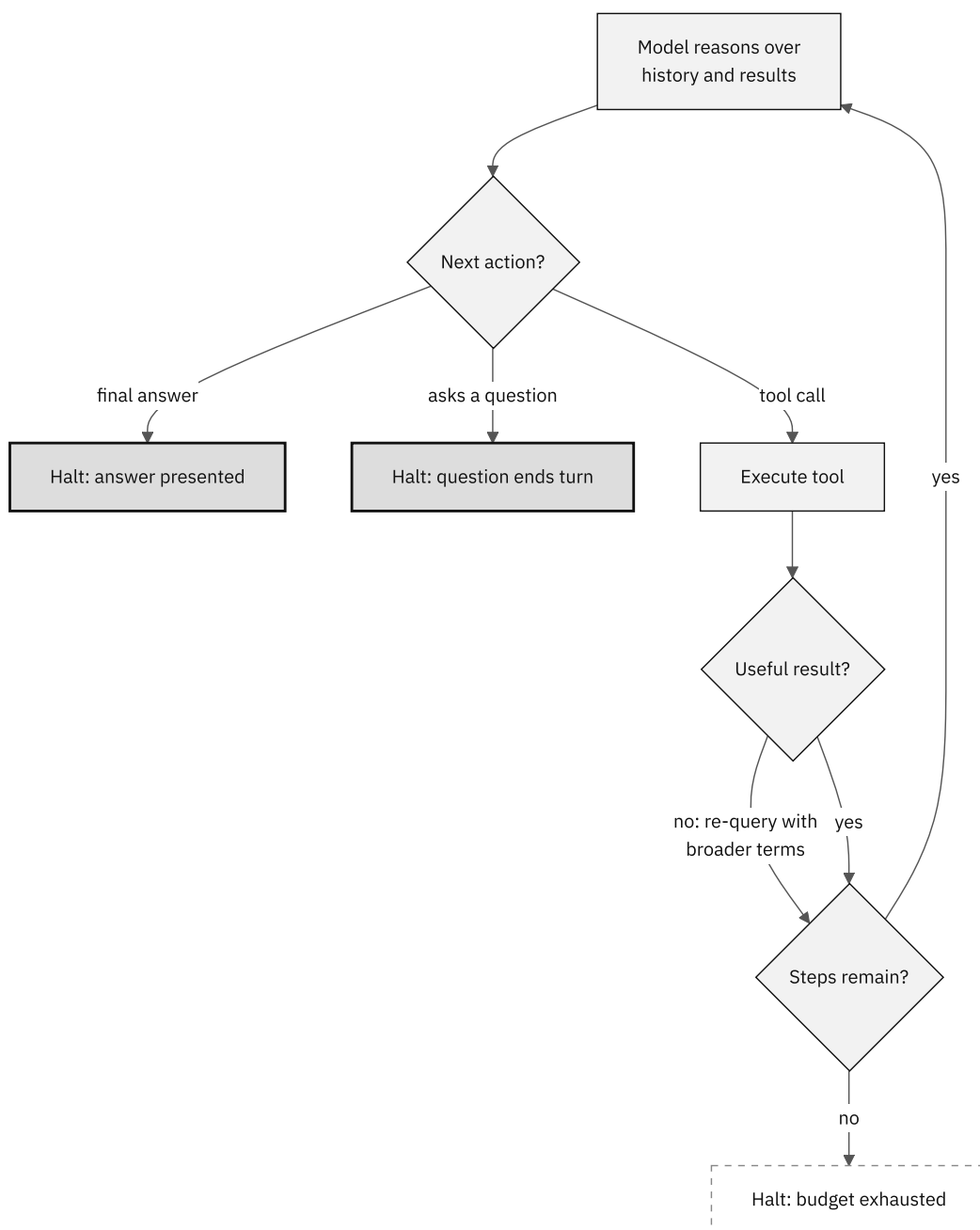
## 5.1 The loop and when it halts

Each turn permits the model up to ten reasoning steps, after which the turn ends regardless. This budget bounds cost and prevents a confused model from looping indefinitely on a failing search.

The second stopping condition is more interesting. When the agent asks the shopper a clarifying question, **the turn ends immediately**. Without this rule the model would ask a question and then continue working, rendering a product carousel underneath its own unanswered question. Making clarification terminal converts a conversational convention into a structural guarantee.

FIGURE 3

The reasoning loop, showing all three exits and the self-correction path back into search.



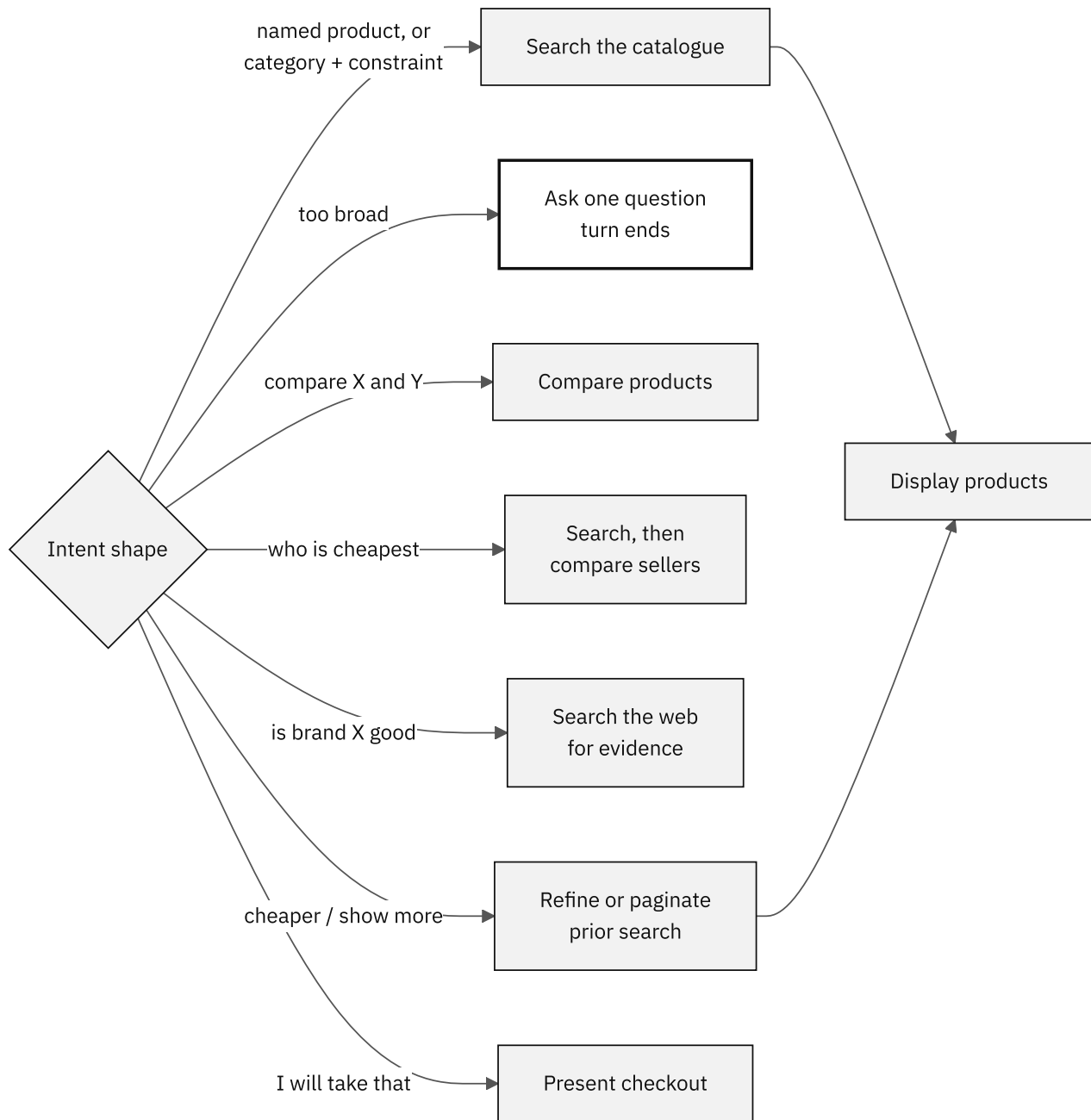
The self-correction edge is policy-driven: the agent must attempt several differentiated queries before it is permitted to report failure.

## 5.2 Decision policy

The agent's planning policy is a decision tree over the shape of the user's utterance. It is written as explicit, testable rules rather than as general encouragement.

FIGURE 4

How the shape of a request selects the agent's opening action.



Only the clarification branch is terminal. Search-shaped branches converge on the single action that can place a card on screen.

### 5.3 Working memory

Follow-up requests such as “cheaper” or “show me more” are meaningless without state. The agent maintains a compact record of the last query, the filters applied, and the identifiers already shown, so that a refinement narrows the previous search rather than starting a new one, and pagination excludes what the shopper has already seen.

The design decision worth defending is that **this memory is never stored separately**. It is reconstructed from the conversation history at the start of every request. Because each



tool result is itself recorded as part of the conversation, the memory is fully recoverable from the transcript. This removes an entire class of bug in which cached state and conversation history disagree, at the cost of a small amount of work per request.

## 5.4 Reproducibility with variation

A demonstration in which two evaluators type the same query and receive byte-identical output appears scripted; one in which the same conversation yields different answers on reload appears broken. The system resolves this by deriving a seed from the conversation identifier and using it to vary the agent's tone, the ordering of lower-ranked results, and interface copy.

The resulting property is that the **same conversation always produces the same output, while different conversations differ in voice but never in fact**. Variation is confined strictly to presentation; it never touches which products exist or what they cost.

---

## 06 Tool design

The agent's eleven tools divide into four groups. Each declares a typed, validated input schema, so malformed calls are rejected before any external request is made.

| GROUP                             | TOOLS                                          | RESPONSIBILITY                                                                                                                    |
|-----------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Discovery</b>                  | Search, Refine, Show more                      | Query the catalogue; refine narrows the prior query, show-more paginates it while excluding items already seen.                   |
| <b>Presentation</b>               | Display products                               | The only action that can place a product card on screen.                                                                          |
| <b>Analysis</b>                   | Get product, Compare products, Compare sellers | Detail on one item, side-by-side comparison of two to four, and every merchant offering one product ordered by price.             |
| <b>Interaction &amp; evidence</b> | Clarify intent, Buy, Web search, Web fetch     | Ask a multiple-choice question, present a checkout route, and gather external evidence for questions the catalogue cannot answer. |

Searching and displaying are deliberately distinct actions. Search results are private working data for the agent to reason over; placing them in front of the shopper requires a second, explicit decision. This is what allows the agent to search three times, discard two result sets as off-target, and present only the third — rather than showing the shopper every attempt.

## 6.1 Ranking

The search tool does more than relay the catalogue's response. It requests more results than it intends to show, so that later stages have slack to discard from; removes products rated below a threshold, while falling back to the unfiltered set if too few survive, so that sparse categories never return empty; applies the requested ordering; and finally holds the strongest results in place while lightly varying the order of the remainder.

The rating filter exists because of an observed failure rather than a hypothetical one: a very poorly rated product ranked first by pure relevance made the entire agent appear broken during testing. The fallback clause exists because the first version of that filter produced empty results in thinly populated categories.

# 07 Integration

## 7.1 The retail catalogue

Product data comes from a cross-merchant retail catalogue exposed through the Model Context Protocol, an emerging standard for giving agents structured access to external systems. This is the integration that makes the project agentic commerce rather than a conversational wrapper over a static dataset.

Three operations are used: keyword search with filters, detail retrieval for a single product, and batch retrieval by identifier used as a fallback. Each request identifies the calling agent and declares its capabilities to the catalogue — the protocol's mechanism for an agent presenting itself to a merchant network.

|               |                                                                                                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prices        | Handled as integer minor units throughout to avoid floating-point error. Budgets given in other currencies are converted before filters are applied.                      |
| Normalisation | Merchants populate the same fields inconsistently. A normalisation layer collapses these variants into one internal shape and discards records too incomplete to display. |

## Error handling

Three distinct failures are separated: transport errors, protocol-level errors, and well-formed responses containing no usable data. Each is surfaced to the agent differently.

## Caching

Deliberately none. The catalogue's terms prohibit caching search results, and freshness is the entire point of using a live catalogue.

### WHY THIS CATALOGUE

The alternative was a single-merchant storefront interface, which would have required seeding a development store with invented inventory and would have limited the agent to one merchant's stock. The cross-merchant catalogue returns genuine purchase routes and lets the agent compare sellers, which is the substance of the problem. The cost is real and visible in the evaluation: coverage is uneven, and some categories return accessories rather than the devices themselves.

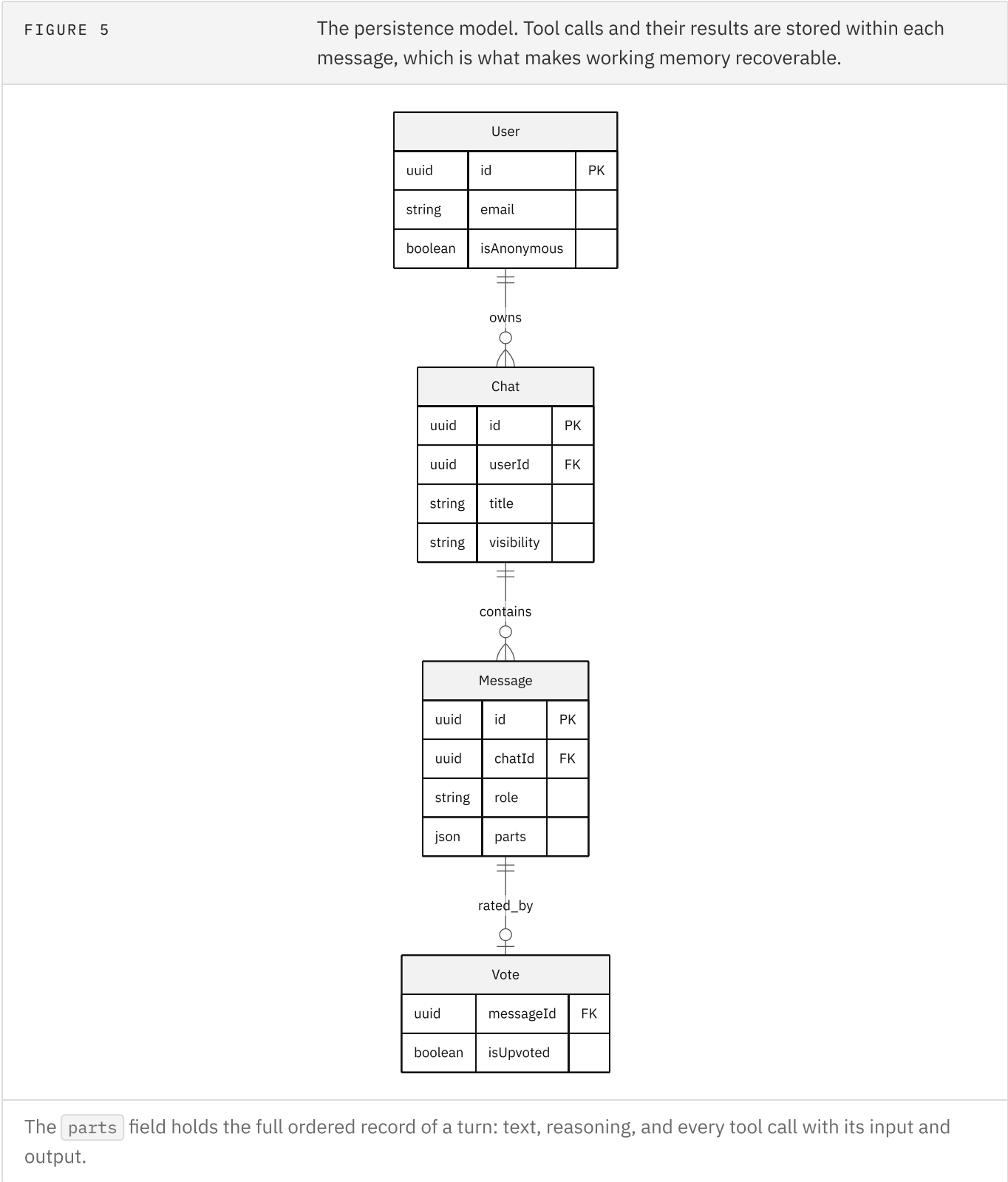
## 7.2 Language models and web evidence

Inference is routed through an aggregator exposing twenty-nine models, of which twenty-two support tool calling. All are free-tier, so the work is reproducible without paid credentials; the trade is rate limiting and weaker instruction-following, both of which appear in the evaluation. The model is selectable at run time, which allows the same agent to be compared across models — a useful property when the research question is how much of the behaviour comes from the policy and how much from the model.

For questions the catalogue cannot answer — brand reliability, expert reviews, compatibility — the agent gathers evidence from web search, with a fallback engine when the primary one fails. The agent is explicitly forbidden from answering such questions from its own knowledge when this evidence is unavailable.

08

# Data and caching

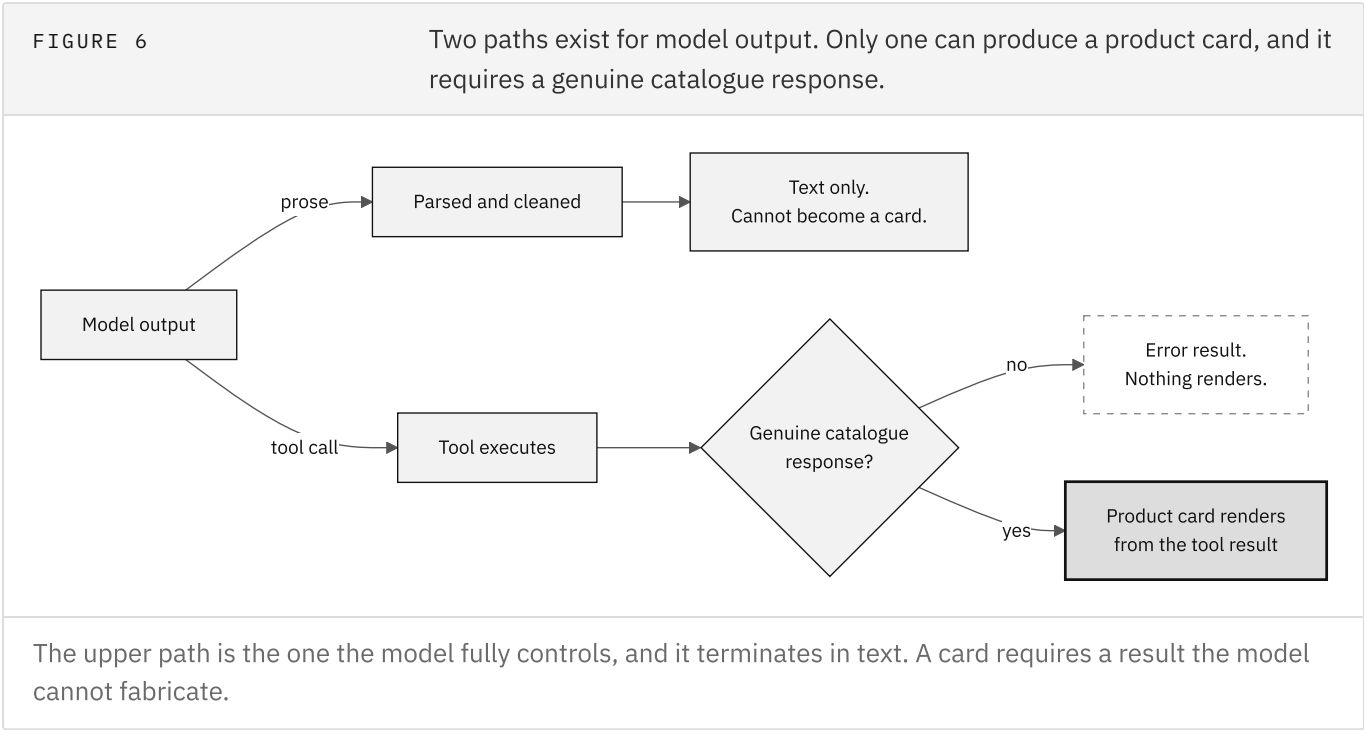


That single storage decision is what makes three other things work. Historical conversations re-render exactly as they originally appeared; working memory can be rebuilt from the transcript; and the evaluation harness can score a turn by reading the sequence of actions it recorded.

Caching is applied only where it is safe. Browsable feed results are cached for an hour, feed searches for five minutes, and web evidence for five minutes within the running process. Catalogue queries made by the agent are never cached. The web-evidence cache is more consequential than it appears: without it, two evidence lookups in one turn would each launch a separate browser instance.

## 09 Grounding

This is the aspect of the system that most deserves scrutiny, because it is where an agentic architecture earns its keep over a well-written prompt.



The structural guarantee holds, but weaker models still violate the *formatting* contract frequently. Rather than assuming prompt engineering had solved this, the interface assumes the model will misbehave and defends against it: it unwraps structured responses even when the model wraps them in extra prose, strips duplicate tables the model was asked not to produce, and recovers a question menu even when the model writes it as text instead of invoking the proper action.

**AN HONEST NOTE**

The third defence is a workaround, not a fix. The evaluation shows two of eight scenarios failing in exactly this way. The interface recovers, so the shopper still sees a usable menu, but the agent's action sequence was wrong and the harness correctly flags it. Reporting those runs as successes because the screen looked right would be the wrong lesson to draw.

Alongside these mechanisms, the agent operates under explicit constraints: ground every product claim in a result from the current turn; never recommend a product not seen in such a result; refuse compatibility questions without external evidence; and always use each product's exact catalogue title.

# 10 Evaluation

Prose quality is the wrong thing to measure in an agent. Behaviour is the right thing. The evaluation harness drives the real system through eight scenarios chosen to exercise distinct branches of the decision policy, and asserts on **which actions the agent took, and in what order**.

## 10.1 Method

Each scenario begins a fresh conversation, so no state carries between them. The harness captures the full response stream, extracts the sequence of actions, and applies assertions derived from the agent's own policy: a broad request must ask a clarifying question first; a comparison request must invoke the comparison action; a “cheapest” request must compare sellers rather than substituting a web search; output must contain no formatting the agent was instructed to avoid.

One methodological rule matters more than the others: when a provider rate limit occurs, the behavioural assertions are suppressed for that run. Without this, ordinary infrastructure flakiness would be recorded as a reasoning defect, and the results would be meaningless.

## 10.2 Results

| SCENARIO TESTS         | REQUEST                               | OUTCOME          |
|------------------------|---------------------------------------|------------------|
| Named product          | find a specific pair of earbuds       | CLEAN            |
| Budget constraint      | running shoes under \$80              | CLEAN            |
| Comparison branch      | compare two e-readers                 | CLEAN            |
| Seller comparison      | cheapest place to buy a mouse         | CLEAN            |
| Category plus use case | laptop for video editing under \$1200 | CLEAN            |
| Broad request          | I need new headphones                 | QUESTION AS TEXT |
| Open-ended gift        | gift for someone who likes cooking    | QUESTION AS TEXT |
|                        |                                       |                  |

**Five of eight scenarios ran cleanly; three were flagged.** The failures are informative rather than embarrassing, and each has a diagnosed cause.

In the two broad-request scenarios the model produced a correctly structured clarifying question but wrote it as ordinary text instead of invoking the question action. The interface's recovery meant the shopper still saw a working menu, but the agent's action sequence was wrong. This is a limit of free-tier instruction-following; the remedy is constrained decoding or a stronger model, not additional prompt text.

In the third, the catalogue returned accessories rather than devices. The agent correctly recognised the results as off-target and said so rather than presenting wrong products — the behaviour the policy requires — but it did not exhaust its retry budget first.

#### WHAT THE HARNESS DEMONSTRATES

An agent evaluated only by reading its replies looks better than it is. Asserting on action sequences surfaced two failures whose output appeared entirely correct on screen. That gap — between plausible output and correct behaviour — is the central evaluation problem in agentic systems, and it is the reason this harness exists.

## 11 Limitations and future work

Stated plainly, because a report claiming a system has no weaknesses is not describing a real system.

| LIMITATION                           | CONSEQUENCE                                                              | STATUS    |
|--------------------------------------|--------------------------------------------------------------------------|-----------|
| No automatic failover between models | A rate limit interrupts the session; the user must switch model by hand. | KNOWN GAP |
| Instruction drift on weaker models   | Two of eight scenarios produced a structured payload as text.            | MITIGATED |
| Web evidence by page scraping        | Slow, and fragile when search engines change their markup.               | FRAGILE   |
| Uneven catalogue coverage            | Some categories return accessories; many merchants omit specifications.  | UPSTREAM  |

### 11.1 What I would build next

1. **Automatic failover between models.** The metadata needed already exists but is unused. Retrying a rate-limited request against the next available model is the highest-value single improvement and closes the most visible gap above.
2. **Constrained output for clarifying questions.** Forcing the model to emit a structured action rather than hoping it complies would fix both evaluation failures at the root, instead of recovering from them in the interface.
3. **A larger evaluation set.** Eight scenarios cover the main branches of the decision policy but not multi-turn refinement chains, which is precisely where working memory is exercised.

---

## 12 Conclusion

Kasparro Shopper demonstrates that the useful part of an agentic system is not the conversation but the constraints around it. The loop, the stopping conditions, the typed tool interface, the derived working memory and the grounding contract are what turn a language model into something that can be relied upon to talk about real products at real prices.

The evaluation supports a more specific claim. Two scenarios produced output that looked entirely correct on screen while the agent's underlying behaviour was wrong, and only assertions over action sequences caught them. For agentic systems, this suggests that behavioural evaluation is not an optional addition to output review, but the primary instrument — and that a system's honesty about its own failures is a better measure of engineering quality than a demonstration in which nothing goes wrong.

#### ACKNOWLEDGEMENT

This report is submitted by Ridhi Kumari. The system was built collaboratively as a two-person project; product framing, the choice of catalogue, the grounding principle and the clarification policy were decided jointly, with implementation divided between the agent service and the interface layer.